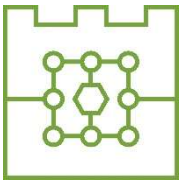


Politechnika Krakowska
im. Tadeusza Kościuszki

Wydział Informatyki i Matematyki



Wiktor Zmiendak

Numer albumu: 142706

Sprawozdanie

Projekt 3

Implementacja algorytmu RSA

Kraków 2026

Cel zadania

Celem projektu była własna implementacja algorytmu RSA oraz zbadanie, jak rośnie trudność faktoryzacji wraz ze wzrostem rozmiaru liczb. W ramach zadania został napisany program w Pythonie tak, aby można go było uruchamiać w dwóch trybach. Pierwszy tryb realizuje pełny proces RSA dla tekstu wczytanego z pliku a więc polega on na podzieleniu wiadomości na bloki, zamianie ich na liczby, szyfrowaniu, odszyfrowaniu i sprawdzeniu poprawności wyników. Drugi tryb służy do faktoryzacji liczb pół pierwszych metodą Pollarda Rho, zapisania wyników do pliku CSV, dopasowania prostych modeli czasowych oraz narysowania wykresu.

Implementacji

Program zaczyna się od definicji trzech alfabetów. Pierwszy alfabet zawiera 26 wielkich liter, drugi rozszerza go o małe litery, cyfry, spację i kropkę, a trzeci obejmuje pełne ASCII. Wybór alfabetu ma znaczenie, ponieważ każdy znak musi zostać jednoznacznie zamieniony na liczbę, zanim wiadomość zostanie zaszyfrowana.

```
ALPHABETS = {
    "26": "ABCDEFGHIJKLMNOPQRSTUVWXYZ",
    "64": "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789 .",
    "ascii": "".join(chr(i) for i in range(128)),
}
```

Następnie wiadomość jest dzielona na bloki o stałej długości przez funkcję *chunks*. W projekcie przyjęto blok o długości 10 znaków, ponieważ taki podział pokazanie działania RSA na małych fragmentach tekstu. Jeśli ostatni blok jest krótszy od wymaganej długości, zostaje uzupełniony pierwszym znakiem wybranego alfabetu.

```
def chunks(text: str, size: int):
    return [text[i : i + size] for i in range(0, len(text), size)]
```

Samą zamianę bloku tekstu na liczbę wykonuje funkcja *block_to_int*. Działa ona jak zapis liczby w systemie pozycyjnym, tylko że podstawą nie jest 10, lecz rozmiar alfabetu. Każdy kolejny znak przesuwa dotychczasową wartość w lewo i dodaje indeks znaku. Odwrotną operację realizuje *int_to_block*, która przez kolejne dzielenie modulo odtwarza znaki z liczby.

```
def block_to_int(block: str, base: int, c2i: dict[str, int]) -> int:
    x = 0
    for ch in block:
        x = x * base + c2i[ch]
    return x
```

Kolejnym etapem jest generowanie liczb pierwszych. Funkcja *is_probable_prime* realizuje test Millera-Rabina, który jest testem probabilistycznym. Funkcja *prime* losuje liczbę o zadanej liczbie bitów, wymusza jej nieparzystość i sprawdza zarówno warunek pierwszości, jak i warunek względnej pierwszości z wykładnikiem „e”.

Najważniejszą częścią jest *rsa_key*, która generuje pełny klucz RSA. Funkcja ta losuje dwie różne liczby pierwsze „p” i „q”, oblicza ich iloczyn „n”, wyznacza funkcję Eulera phi = (p - 1)(q - 1), sprawdza warunek gcd(e, phi) = 1, a następnie oblicza odwrotność modularną „d”. W wyniku zwracany jest słownik zawierający te wartości.

```python
def rsa_key(bits: int = 768, e: int = 65537):
    if bits < 768:
        raise ValueError("Klucz RSA musi miec >= 768 bitow")

    a = bits // 2
    b = bits - a
    while True:
        p = prime(a, e)
        q = prime(b, e)
        if p == q:
            continue
        n = p * q
        if n.bit_length() < bits:
            continue
        phi = (p - 1) * (q - 1)
        if math.gcd(e, phi) != 1:
            continue
        d = pow(e, -1, phi)
        return {"p": p, "q": q, "n": n, "e": e, "d": d}
```

Cały proces szyfrowania i odszyfrowywania realizuje funkcja *run_rsa*. Najpierw wczytuje plik wejściowy, sprawdza alfabet, dzieli tekst na bloki, uzupełnia je do pełnej długości i zamienia każdy blok na liczbę całkowitą. Następnie generuje klucz RSA i dla każdego bloku oblicza szyfrogram według wzoru $c = m^e \bmod n$. Po zaszyfrowaniu program od razu wykonuje odszyfrowanie, licząc $m = c^d \bmod n$, a później zamienia liczby z powrotem na tekst. Ostatecznie odszyfrowany tekst jest porównywany blok po bloku z tekstem oryginalnym.

Drugi tryb programu dotyczy faktoryzacji. Funkcja *pollard_rho_once* wykonuje jedną próbę rozkładu liczby z użyciem metody Pollarda Rho. Algorytm opiera się na generowaniu pseudolosowego ciągu i szukaniu dzielnika przez obliczanie największego wspólnego dzielnika różnicy kolejnych wartości z liczbą faktoryzowaną. Funkcja *factor_pollard_rho* powtarza tę próbę wielokrotnie, aby zwiększyć szansę powodzenia.

```python
def pollard_rho_once(n: int, max_iters: int = 2_000_000):
    if n % 2 == 0:
        return 2, 1

    x = random.randrange(2, n - 1)
    y = x
    c = random.randrange(1, n - 1)
    d, it = 1, 0

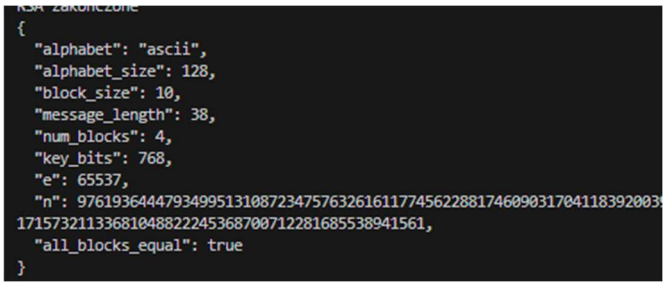
    while d == 1 and it < max_iters:
        x = (x * x + c) % n
        y = (y * y + c) % n
        y = (y * y + c) % n
        d = math.gcd(abs(x - y), n)
        it += 1

    if d in (1, n):
        return None, it
    return d, it
```

Do samego benchmarku służy funkcja *semiprime*, która generuje liczbę będącą iloczynem dwóch liczb pierwszych o zadanej liczbie bitów. Funkcja *run_factor* uruchamia cały benchmark dla listy rozmiarów kluczy, mierzy czas działania, zapisuje wyniki do CSV, dopasowuje modele i tworzy wykres.

Wyniki uzyskane w uruchomieniu

W uruchomionym eksperymencie użyto alfabetu ascii, rozmiaru bloku 10 znaków oraz klucza o długości 768 bitów. Wiadomość wejściowa miała 38 znaków i została podzielona na 4 bloki. Wygenerowany klucz miał wykładnik publiczny `e = 65537`, a jego moduł „n” był liczbą 768-bitową.



Uzasadnienie wyboru wartości „e”

W projekcie jako standardową wartość wykładnika publicznego przyjęto e = 65537. Jest to jeden z najczęściej stosowanych wyborów w praktyce kryptograficznej, ponieważ łączy w sobie bezpieczeństwo i wydajność. Wartość ta jest liczbą pierwszą i ma bardzo prosty zapis binarny, równy 10000000000000001. Oznacza to, że potęgowanie wymaga stosunkowo małej liczby mnożeń, więc szyfrowanie jest szybkie.

Dlaczego RSA wymaga warunku m < n

Warunek m < n jest konieczny, ponieważ RSA działa modulo „n”, a więc każda wiadomość jest najpierw redukowana do klasy reszt modulo „n”. Jeśli blok ma wartość większą lub równą „n”, to w praktyce zostaje zastąpiony przez swoją resztę z dzielenia przez „n”. W takim przypadku deszyfrowanie nie odtworzy pierwotnej wiadomości, lecz jedynie wartość m mod n.

```python
# Generowanie klucza RSA
key = rsa_key(bits=key_bits)

encrypted = []
for m in values:
    if m >= key["n"]:
        raise ValueError("Blok musi spelnic warunek m < n")
    encrypted.append(pow(m, key["e"], key["n"]))
```

Dlaczego identyczne bloki dają identyczne szyfrogramy

W standardowym algorytmie RSA szyfrowanie jest deterministyczne. Oznacza to, że dla tej samej wiadomości „m” i tego samego klucza publicznego zawsze otrzymuje się ten sam szyfrogram „c”. Jeśli dwa bloki tekstu są identyczne, to po zamianie na liczby również są identyczne, a więc po wykonaniu $c = m^e \bmod n$ otrzymujemy dokładnie ten sam wynik.

Czym jest padding i jak rozwiązuje problem

Padding to dodatkowe dane wprowadzane do wiadomości przed szyfrowaniem. Jego zadaniem jest zmiana wejścia w taki sposób, aby ten sam tekst jawny nie dawał zawsze tego samego szyfrogramu. Najczęściej padding zawiera losowy składnik, dzięki czemu szyfrowanie staje się niedeterministyczne.

Jak rośnie trudność faktoryzacji wraz z rozmiarem „n”

Wyniki benchmarku pokazują bardzo wyraźny wzrost trudności faktoryzacji wraz z rozmiarem liczby. Dla 32 bitów metoda Pollarda Rho znalazła czynniki w około 0,00012 sekundy, a dla 40 bitów w około 0,001 sekundy. Dla 56 bitów czas wzrósł już do około 0,0046 sekundy, dla 64 bitów do około 0,018 sekundy, dla 72 bitów do około 0,096 sekundy, dla 80 bitów do około 0,835 sekundy, a dla 88 bitów do około 6,06 sekundy. Wraz ze wzrostem liczby bitów rosną także liczby iteracji, co potwierdza, że zadanie faktoryzacji bardzo szybko staje się trudniejsze.

W praktyce oznacza to, że nawet jeśli małe liczby można rozkładać bardzo szybko, to dla dużych modułów RSA czas ataku zaczyna rosnąć skrajnie niekorzystnie. Klucz o długości 768 bitów nadal jest znacznie trudniejszy do złamania niż liczby testowane w benchmarku, a standardowe klucze 2048-bitowe są obecnie uznawane za praktycznie bezpieczne wobec takich prostych metod faktoryzacji.

Dlaczego wybrano właśnie taki typ funkcji dopasowania

W programie dopasowano trzy typy modeli: liniowy, potęgowy i wykładniczy. Spośród rozważanych funkcji najlepiej do danych pasuje model wykładniczy. Wybór tego modelu jest uzasadniony także merytorycznie. Sama metoda Pollarda Rho nie musi w sensie ścisłym rosnąć wykładniczo dla wszystkich zakresów, ale przy tak małym zbiorze pomiarów i w praktycznym zakresie bitów obserwuje się bardzo szybki wzrost kosztu obliczeniowego. Model wykładniczy najlepiej oddaje to przyspieszenie wzrostu w porównaniu z prostą funkcją liniową.

Czy można ufać tym prognozom dla dużych kluczy

Prognozom dla bardzo dużych kluczy, takich jak 1024, 2048 czy 4096 bitów, nie można ufać bez zastrzeżeń. Ekstrapolacja została wykonana na podstawie bardzo małego zakresu danych, obejmującego tylko liczby 32–88 bitów. Oznacza to, że model został dopasowany do krótkiego fragmentu skali i może zachowywać się zupełnie inaczej poza zakresem pomiarów.